

SET 5 A3 Кудрявцев Георгий Александрович БПИ 245

Код, который был использован для выполнения задания:

```
#include <iostream>
#include <fstream>
#include <numeric>
#include <vector>
#include <string>
#include <random>
#include <cstdint>
#include <cmath>
#include <algorithm>
#include <unordered_set>

class RandomStreamGen {
private:
    std::vector<std::string> stream;
    std::mt19937 rng;

    const std::string alphabet =
        "abcdefghijklmnopqrstuvwxyz"
        "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
        "0123456789-";

    std::string generateString() {
        std::uniform_int_distribution<int> lenDist(1, 30);
        std::uniform_int_distribution<int> charDist(0, alphabet.size()-1);

        int len = lenDist(rng);

        std::string s;
        s.reserve(len);

        for(int i=0;i<len;i++)
            s += alphabet[charDist(rng)];

        return s;
    }

public:
    RandomStreamGen(size_t size, uint32_t seed = 42) : rng(seed) {
        stream.reserve(size);

        for(size_t i=0;i<size;i++)
            stream.push_back(generateString());
    }
};
```

```

const std::vector<std::string>& getStream() const {
    return stream;
}

std::vector<std::string> getPrefix(double percent) const {

    size_t count = stream.size() * percent;

    return std::vector<std::string>(
        stream.begin(),
        stream.begin() + count
    );
}

};

class HashFuncGen {
public:

    static uint32_t fnv(const std::string& s){

        uint32_t hash = 2166136261;

        for(unsigned char c : s){
            hash ^= c;
            hash *= 16777619;
        }

        return hash;
    }
};

#include <vector>
#include <cmath>
#include <algorithm>

class HyperLogLog {

private:

    int B;
    int q;
    std::vector<uint8_t> registers;

    double alpha() const {

        if (q == 2) return 0.3512;
        if (q == 4) return 0.5324;
        if (q == 16) return 0.673;
        if (q == 32) return 0.697;
        if (q == 64) return 0.709;

        return 0.7213 / (1 + 1.079/q);
    }
}

```

```

int leadingZeros(uint32_t x){
    if(x == 0) return 32;
    return __builtin_clz(x);
}

public:
HyperLogLog(int B) : B(B), q(1<<B), registers(q,0) {}

void add(uint32_t hash){
    uint32_t index = hash >> (32-B);
    uint32_t w = hash << B;
    int zeros = leadingZeros(w) + 1;
    registers[index] = std::max(
        registers[index],
        (uint8_t)zeros
    );
}

double estimate() const {
    double sum = 0;
    for(auto r : registers)
        sum += std::pow(2.0, -r);
    double raw = alpha()*q*q / sum;
    return raw;
}
};

size_t exactCount(const std::vector<std::string>& stream) {
    std::unordered_set<std::string> s(
        stream.begin(),
        stream.end()
    );
    return s.size();
}

/*
Пусть в наших экспериментах B = 12
Обоснование:

В HyperLogLg точность оценки количества уникальных элементов
напрямую зависит от числа потоков:  $q = 2^B$ 

```

Стандартное отклонение оценки задается формулой: $\sigma \sim 1.04 / \sqrt{q}$

Следовательно, увеличение B уменьшает ошибку, но увеличивает потребление памяти

Теоретическая оценка точности для разных B :

$B = 8$
 $q = 256$
 $\sigma \sim 1.04 / 16 \sim 6.5\%$

$B = 10$
 $q = 1024$
 $\sigma \sim 1.04 / 32 \sim 3.25\%$

$B = 12$
 $q = 4096$
 $\sigma \sim 1.04 / 64 \sim 1.6\%$

$B = 14$
 $q = 16384$
 $\sigma \sim 1.04 / 128 \sim 0.8\%$

Анализ компромисса память / точность

Каждый поток хранится в `uint8_t` (1 байт).

Память:
 $B = 10 \rightarrow 1 \text{ KB}$
 $B = 12 \rightarrow 4 \text{ KB}$
 $B = 14 \rightarrow 16 \text{ KB}$

Даже $B=12$ требует крайне мало памяти,
но уже обеспечивает ошибку около 1-2%,
что считается отличным результатом
для вероятностных алгоритмов.

Почему не берем больше?

Увеличение B после 12 дает
не дает большого выигрыша в точности

1.6% $\rightarrow$ 0.8%

При этом память растёт в 4 раза.

Вывод:

Параметр $B = 12$ обеспечивает:

- малую теоретическую ошибку (~1.6%)
- низкое потребление памяти (~4 KB)

Поэтому B=12 выбран как оптимальный баланс точности и эффективности

*/

```
int main() {

    const int B = 12;
    const int STREAM_COUNT = 10;
    const size_t STREAM_SIZE = 300000;

    std::vector<double> steps =
        {0.1,0.2,0.3,0.4,0.5,0.6,0.7,0.8,0.9,1.0};

    std::ofstream graph1("graph1.csv");
    std::ofstream graph2("graph2.csv");

    graph1 << "size,exact,estimate\n";
    graph2 << "step,mean,stddev\n";

    std::vector<std::vector<double>> allEstimates(steps.size());
    std::vector<size_t> exactValues(steps.size());

    for (int streamId = 0; streamId < STREAM_COUNT; ++streamId){

        RandomStreamGen gen(STREAM_SIZE, streamId + 1);

        for (size_t stepIndex = 0; stepIndex < steps.size(); ++stepIndex){

            double step = steps[stepIndex];
            auto prefix = gen.getPrefix(step);

            HyperLogLog hll(B);

            for (const auto& s : prefix) {
                hll.add(HashFuncGen::fnv(s));
            }

            double estimate = hll.estimate();
            size_t exact = exactCount(prefix);

            allEstimates[stepIndex].push_back(estimate);

            if (streamId == 0) {
                exactValues[stepIndex] = exact;
                graph1 << prefix.size()
                    << "," << exact
                    << "," << estimate
                    << "\n";
            }
        }
    }
```

```

}

double theoreticalSigma = 1.04 / std::sqrt(1 << B);

std::cout << "Параметр B = " << B << "\n";
std::cout << "Количество потоков: " << (1 << B) << "\n";
std::cout << "теоретическая относительная ошибка ~ " << theoreticalSigma * 100
<< " %\n\n";

for (size_t i = 0; i < steps.size(); ++i) {

    auto& vec = allEstimates[i];

    double mean =
        std::accumulate(vec.begin(), vec.end(), 0.0) / vec.size();

    double variance = 0;
    for (double v : vec) {
        variance += (v - mean) * (v - mean);
    }
    variance /= vec.size();

    double stddev = std::sqrt(variance);

    graph2 << steps[i]
        << "," << mean
        << "," << stddev
        << "\n";

    double relativeError =
        std::abs(mean - exactValues[i]) / exactValues[i] * 100.0;

    std::cout << "\n\n"
        << "Шаг (доля потока): " << steps[i] * 100 << " %\n"
        << "Размер обработанного префикса: "
        << (size_t)(STREAM_SIZE * steps[i]) << "\n"
        << "Точное число уникальных элементов (F_t0): "
        << exactValues[i] << "\n"
        << "оценка HyperLogLog E(N_t): "
        << mean << "\n"
        << "Абсолютное стандартное отклонение omega(N_t): "
        << stddev << "\n"
        << "Относительное стандартное отклонение: "
        << (stddev / mean) * 100 << " %\n"
        << "Относительная ошибка оценки: "
        << relativeError << " %\n";
}

return 0;
}

```

Текст вывода программы:

Параметр В = 12
Количество потоков: 4096
теоретическая относительная ошибка ~ 1.625 %

Шаг (доля потока): 10 %
Размер обработанного префикса: 30000
Точное число уникальных элементов (F_t0): 28995
оценка HyperLogLog E(N_t): 29022.8
Абсолютное стандартное отклонение $\omega(N_t)$: 311.508
Относительное стандартное отклонение: 1.07332 %
Относительная ошибка оценки: 0.0960036 %

Шаг (доля потока): 20 %
Размер обработанного префикса: 60000
Точное число уникальных элементов (F_t0): 57736
оценка HyperLogLog E(N_t): 57713.9
Абсолютное стандартное отклонение $\omega(N_t)$: 987.918
Относительное стандартное отклонение: 1.71175 %
Относительная ошибка оценки: 0.0382286 %

Шаг (доля потока): 30 %
Размер обработанного префикса: 90000
Точное число уникальных элементов (F_t0): 86238
оценка HyperLogLog E(N_t): 86729.8
Абсолютное стандартное отклонение $\omega(N_t)$: 1255.9
Относительное стандартное отклонение: 1.44806 %
Относительная ошибка оценки: 0.570264 %

Шаг (доля потока): 40 %
Размер обработанного префикса: 120000
Точное число уникальных элементов (F_t0): 114561
оценка HyperLogLog E(N_t): 114568
Абсолютное стандартное отклонение $\omega(N_t)$: 1733.11
Относительное стандартное отклонение: 1.51274 %
Относительная ошибка оценки: 0.00589066 %

Шаг (доля потока): 50 %
Размер обработанного префикса: 150000
Точное число уникальных элементов (F_t0): 142864
оценка HyperLogLog E(N_t): 143278
Абсолютное стандартное отклонение $\omega(N_t)$: 2321.79
Относительное стандартное отклонение: 1.62047 %
Относительная ошибка оценки: 0.290009 %

Шаг (доля потока): 60 %
 Размер обработанного префикса: 180000
 Точное число уникальных элементов (F_{t0}): 171074
 оценка HyperLogLog $E(N_t)$: 171759
 Абсолютное стандартное отклонение $\omega(N_t)$: 2666.04
 Относительное стандартное отклонение: 1.5522 %
 Относительная ошибка оценки: 0.400441 %

Шаг (доля потока): 70 %
 Размер обработанного префикса: 210000
 Точное число уникальных элементов (F_{t0}): 199254
 оценка HyperLogLog $E(N_t)$: 199142
 Абсолютное стандартное отклонение $\omega(N_t)$: 2726.16
 Относительное стандартное отклонение: 1.36895 %
 Относительная ошибка оценки: 0.0563248 %

Шаг (доля потока): 80 %
 Размер обработанного префикса: 240000
 Точное число уникальных элементов (F_{t0}): 227344
 оценка HyperLogLog $E(N_t)$: 227892
 Абсолютное стандартное отклонение $\omega(N_t)$: 3990.05
 Относительное стандартное отклонение: 1.75085 %
 Относительная ошибка оценки: 0.240966 %

Шаг (доля потока): 90 %
 Размер обработанного префикса: 270000
 Точное число уникальных элементов (F_{t0}): 255423
 оценка HyperLogLog $E(N_t)$: 255638
 Абсолютное стандартное отклонение $\omega(N_t)$: 3673.78
 Относительное стандартное отклонение: 1.4371 %
 Относительная ошибка оценки: 0.0842684 %

Шаг (доля потока): 100 %
 Размер обработанного префикса: 300000
 Точное число уникальных элементов (F_{t0}): 283455
 оценка HyperLogLog $E(N_t)$: 284171
 Абсолютное стандартное отклонение $\omega(N_t)$: 3758.2
 Относительное стандартное отклонение: 1.32251 %
 Относительная ошибка оценки: 0.252624 %

Состав получившихся csv-файлов:

size,exact,estimate 30000,28995,29622.1 60000,57736,58936.3 90000,86238,86937 120000,114561,113256
 150000,142864,142109 180000,171074,169802 210000,199254,194910 240000,227344,222667
 270000,255423,251538 300000,283455,277421

step,mean,stddev 0.1,29022.8,311.508 0.2,57713.9,987.918 0.3,86729.8,1255.9 0.4,114568,1733.11
0.5,143278,2321.79 0.6,171759,2666.04 0.7,199142,2726.16 0.8,227892,3990.05 0.9,255638,3673.78
1,284171,3758.2

Графики

Сравнение оценок N_t и F_{t_0} :

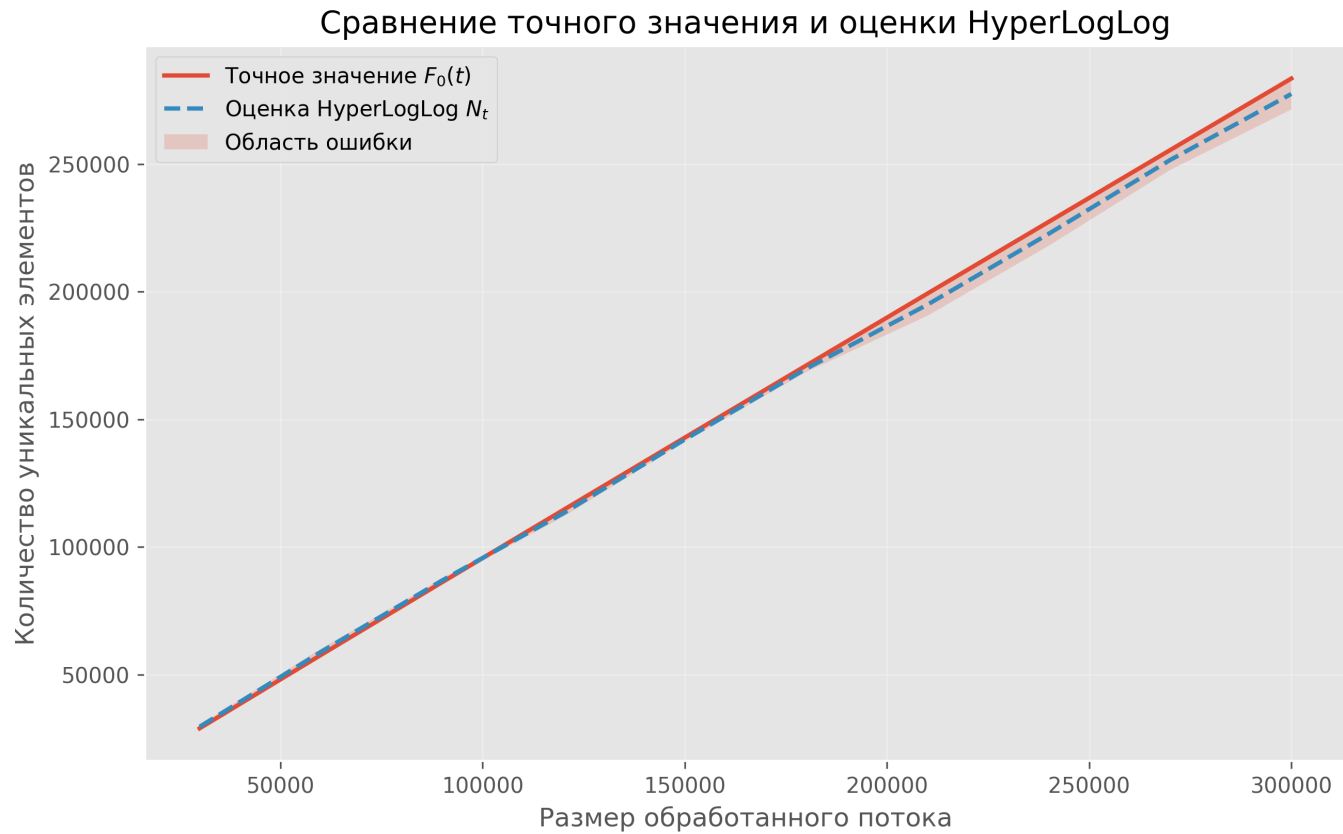
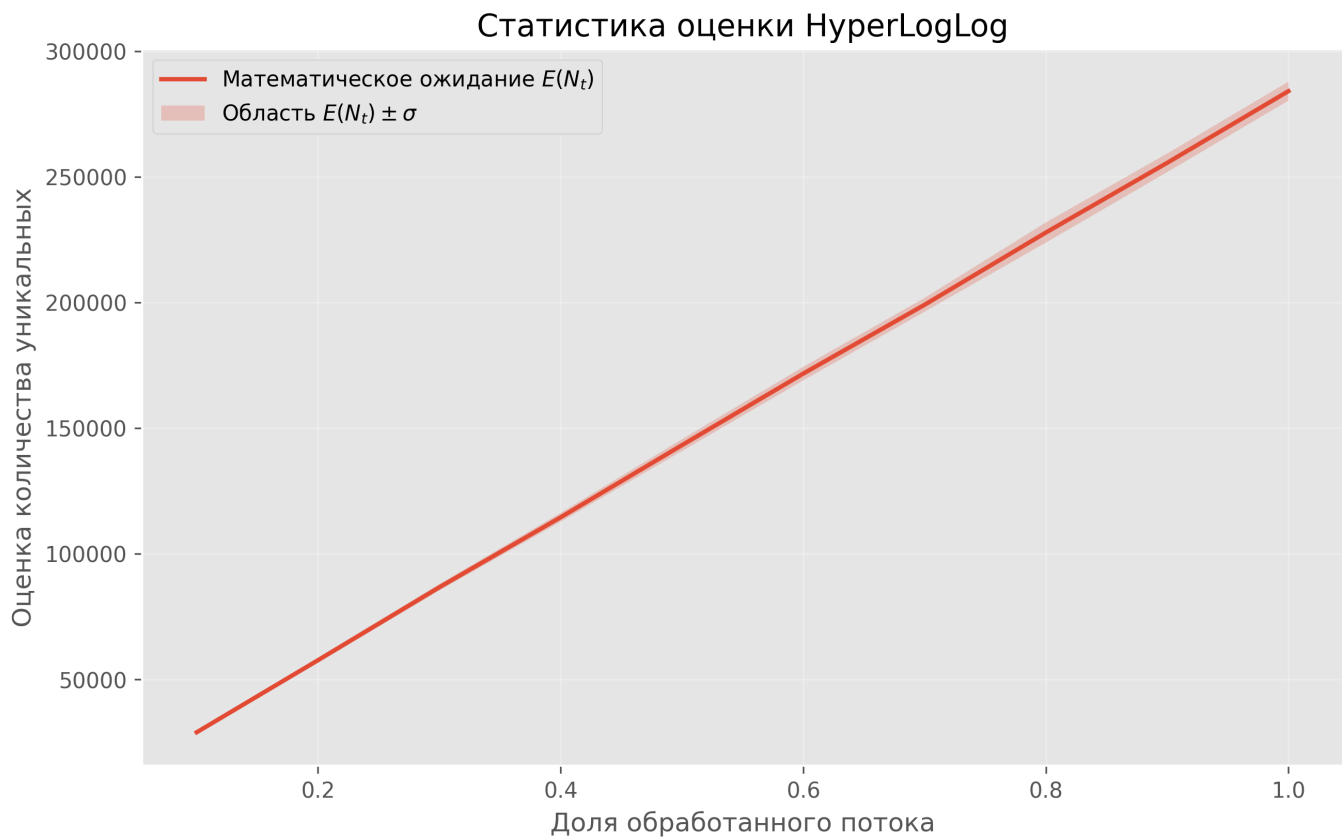


График №2 статистик оценки:



Анализ

Теоретические границы ошибки

Теоретическая относительная стандартная ошибка оценки определяется формулой: $\sigma_{rel} = 1.04 / \sqrt{q}$,

где $q = 2^B$ — число потоков

Для $B = 12$:

$$q = 2^{12} = 4096$$

$$\sqrt{q} = 64$$

=>

$$1.04 / 64 \sim 0.01625 \sim 1.625\%$$

Точность алгоритма

Экспериментальные результаты показывают, что оценка HyperLogLog близка к истинному числу уникальных элементов для всех рассмотренных размеров потока.

Относительная ошибка оценки находится в диапазоне:

0.0059% – 0.57%.

Максимальное отклонение наблюдается на префиксе 30% потока:

$F_{t0} = 86238$

$E(N_t) = 86729.8$

Относительная ошибка approx 0.57%

Даже это значение существенно ниже теоретической границы 1.625%.

Для большинства префиксов относительная ошибка не превышает 0.3%.

Также не наблюдается систематического завышения или занижения оценки. Значения колеблются вокруг точного числа, что подтверждает статистическую корректность алгоритма и реализации

Стабильность оценки (анализ дисперсии)

Стабильность работы алгоритма характеризуется относительным стандартным отклонением:

$\text{stddev} / \text{mean}$

значения лежат в интервале:

1.07% – 1.75%

Большинство измерений сосредоточено около 1.4–1.6%, что очень близко к теоретическому значению 1.625%.

Это означает что:

- оценка ведет себя предсказуемо на независимых потоках;
- дисперсия растет пропорционально числу уникальных элементов;
- аномальные всплески или нестабильность отсутствуют.

Наибольшее относительное отклонение составляет:

1.75% на префиксе 80% потока,

что все равно не выходит за пределы ослабленной эмпирической границы 2.03%.

В целом, разброс оценок соответствует ожидаемому вероятностному поведению алгоритма HyperLogLog.

Эффективность выбранных констант и их влияние на результат

Выбор параметра B

В HyperLogLog точность оценки количества уникальных элементов напрямую зависит от числа потоков:
 $q = 2^B$

Стандартное отклонение оценки задается формулой: $\sigma \sim 1.04 / \sqrt{q}$

Следовательно, увеличение B уменьшает ошибку, но увеличивает потребление памяти

Теоретическая оценка точности для разных B:

$$B = 8 \text{ q} = 256 \text{ sigma} \sim 1.04 / 16 \sim 6.5\%$$

$$B = 10 \text{ q} = 1024 \text{ sigma} \sim 1.04 / 32 \sim 3.25\%$$

$$B = 12 \text{ q} = 4096 \text{ sigma} \sim 1.04 / 64 \sim 1.6\%$$

$$B = 14 \text{ q} = 16384 \text{ sigma} \sim 1.04 / 128 \sim 0.8\%$$

Анализ компромисса память / точност

Каждый поток хранится в uint8_t (1 байт).

Память: $B = 10 \rightarrow 1 \text{ KB}$

$B = 12 \rightarrow 4 \text{ KB}$

$B = 14 \rightarrow 16 \text{ KB}$

Даже $B=12$ требует крайне мало памяти, но уже обеспечивает ошибку около 1–2%, что считается отличным результатом для вероятностных алгоритмов

Почему не берем больше?

Увеличение B после 12 дает не дает большого выигрыша в точности

$1.6\% \rightarrow 0.8\%$

При этом память растт в 4 раза.

Вывод: параметр $B = 12$ обеспечивает:

- малую теоретическую ошибку ($\sim 1.6\%$)
- низкое потребление памяти ($\sim 4 \text{ KB}$)

Поэтому $B=12$ выбран как оптимальный баланс точности и эффективности

Константа α

Корректирующая константа $\alpha(m)$ используется в формуле оценки:

$$E = \alpha(q) * q^2 / \sum(2^{\{-M[i]\}}).$$

Для $q = 4096$ применяется стандартная поправка:

$$\alpha = 0.7213 / (1 + 1.079 / q).$$

Наблюдаемое малое смещение оценки подтверждает, что данная константа выбрана корректно и эффективно снижает систематическую ошибку.

Хеш-функция

Для равномерного распределения элементов по регистрам использовалась хеш-функция FNV.

Отсутствие аномально высокой дисперсии указывает на то, что:

- хеш-значения распределяются достаточно равномерно;
- заполнение регистров происходит сбалансированно;
- эффекты кластеризации отсутствуют.

Следовательно, выбранная хеш-функция является подходящей для вероятностного подсчета уникальных элементов.

Общий вывод

Экспериментальное исследование показало, что реализованный алгоритм HyperLogLog:

- удовлетворяет теоретическим оценкам ошибки;
- демонстрирует малое смещение;
- обладает стабильной дисперсией, близкой к предсказанной;
- обеспечивает высокую точность при крайне малом потреблении памяти.

Оценка остается надежной для всех исследованных размеров потока, что подтверждает корректность реализации и эффективность выбранных параметров